

Normalizing Flow Models

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

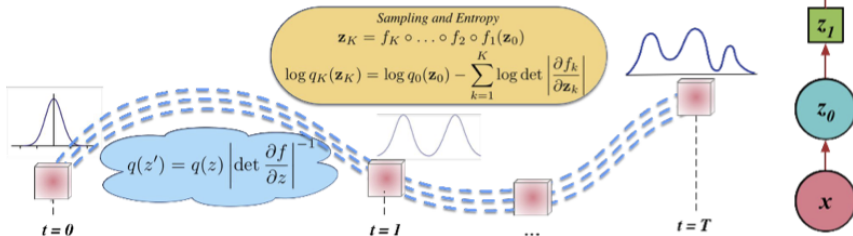
1. Motivation
2. Learning Outcomes
3. Introduction to Normalizing Flows
4. Change of Variables
5. Jacobian
6. Determinant
7. Generators
8. Normalizing Flow Models
9. Learning and Inference
10. Requirements
11. Coupling Layers

- 12. NICE - Non-linear Independent Components Estimation
- 13. RealNVP - Real-valued Non-Volume Preserving
- 14. Glow - Generative Flow
- 15. Flow Matching - Continuous Flows and the Current State of the Art
- 16. Summary
- 17. Limitations
- 18. References

Normalising Flows

Exploit the rule for change of variables:

- Begin with an initial distribution
- Apply a sequence of K invertible transforms



Distribution flows through a sequence of invertible transforms

Rezende and Mohamed, 2015

- ▶ Probabilistic modeling is key to understanding data
- ▶ We want **flexible** models that can:
 - Generate data
 - Compute exact likelihoods
- ▶ Existing models trade off flexibility and tractability:
 - VAEs: Easy to train, blurry generations
 - GANs (generative adversarial networks, covered later): Realistic samples, no likelihoods
 - Normalizing Flows: Both!

1. Understand the concept of normalizing flows.

- 1.1 Learn how normalizing flows transform simple probability distributions into complex ones using invertible mappings.

2. Explore the mathematical foundations.

- 2.1 Study the change of variables formula in probability.
- 2.2 Analyze the role of the Jacobian matrix and its determinant in density estimation.

3. Examine key architectures and training objectives.

- 3.1 NICE (Non-linear Independent Components Estimation)
- 3.2 RealNVP (Real-valued Non-Volume Preserving)
- 3.3 Glow (Generative Flow with Invertible 1x1 Convolutions)
- 3.4 Flow matching (continuous-time flows and the current state of the art)

4. Discuss practical applications and summarize key takeaways.

- 4.1 Review applications in generative modeling and density estimation.
- 4.2 Highlight the main insights and lessons from this topic.

By the end of this session, you will be able to:

1. Understand what Normalizing Flows are and why they matter
2. Apply change of variables in probability
3. Understand the role of Jacobians and determinants
4. Know how flow-based models like NICE, RealNVP, and Glow work
5. Explain how flow matching trains a continuous-time flow by regression, and where it is used today
6. Evaluate benefits and limitations of normalizing flows

Normalizing Flow Models:

Introduction

We continue on our quest for likelihood-based generative models.

So far, we have studied two different types of generative models:

► **Autoregressive Models:** $p_{\theta}(x) = \prod_{i=1}^N p_{\theta}(x_i | x_{<i})$

- Provide tractable likelihoods
- But have no direct mechanism for learning features
- Slow generation process

► **Variational Autoencoders:** $p_{\theta}(x) = \int_z p_{\theta}(x|z)p_{\theta}(z) dz$

- Has intractable marginal likelihood
- Can learn feature representation
- We optimize the lower bound instead of maximizing the likelihood ... we don't know the gap between them

- **Question:** Can we design a latent variable model with tractable likelihoods?

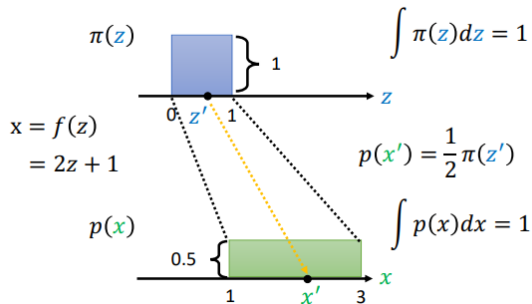
- ▶ **Question:** Can we design a latent variable model with tractable likelihoods?
- ▶ **Answer:** Normalizing Flow Models
 - They combine the best of both worlds, allowing both feature learning and tractable marginal likelihood estimation.
 - **Key Idea:** we wish to map simple distributions (easy to sample and evaluate densities) to complex ones (learned via data) using **change of variables**.

- ▶ A **normalizing flow** is a series of invertible transformations.
- ▶ It maps simple distributions (e.g., Gaussian) into complex ones.
- ▶ Let's say:
 - Sample z from a simple distribution.
 - Transform it to $x = f(z)$.
 - Want $p(x)$, i.e., how likely is x ?

Normalizing Flow Models:

Change of Variables Theorem

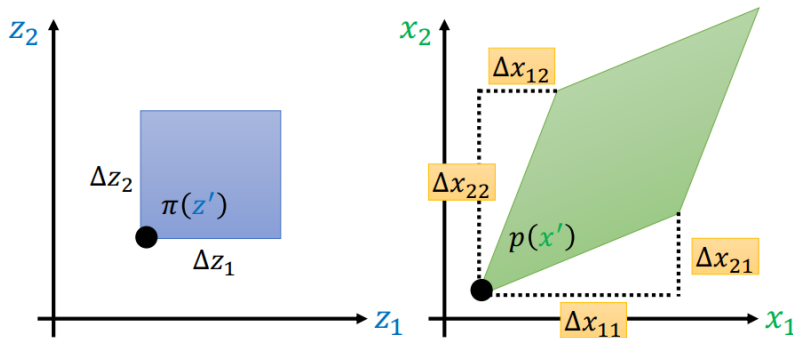
The change of variables formula describes how to evaluate densities of a random variable that is a deterministic transformation of another variable.



Transformation of a 1-D random variable to another random variable. Note that the areas of the blue and green rectangles are equal.

Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 1, slide 12; after Hung-yi Lee, "Flow-based Generative Model" (NTU)

Change of Variables Theorem (cont.)



Transformation of a 2-D random variable. The blue square maps to a green parallelogram; the areas differ by the Jacobian determinant, while the probability masses are equal.

Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 1, slide 23; after Hung-yi Lee, "Flow-based Generative Model" (NTU)

Theorem: Let Z and X be random variables related by a mapping $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ such that $X = f(Z)$ and $Z = f^{-1}(X)$. Then,

$$p_X(x) = p_Z(f^{-1}(x)) \left| \det \left(\frac{\partial f^{-1}(x)}{\partial x} \right) \right|$$

Some important points to note:

- ▶ x and z must be continuous random variables of the same dimension.
- ▶ $\frac{\partial f^{-1}(x)}{\partial x}$ is the $n \times n$ **Jacobian matrix**, where the entry at position (i, j) is $\frac{\partial [f^{-1}(x)]_i}{\partial x_j}$.
- ▶ For any invertible matrix A , $\det(A^{-1}) = [\det(A)]^{-1}$. Therefore, for $z = f^{-1}(x)$,

$$p_X(x) = p_Z(z) \left| \det \left(\frac{\partial f(z)}{\partial z} \right) \right|^{-1}$$

We need:

- ▶ **Invertibility of f :** The function f must be invertible so that for every x there exists a unique $z = f^{-1}(x)$.
- ▶ **Jacobian of f :** The Jacobian matrix $\frac{\partial f(z)}{\partial z}$ must be computable and its determinant non-zero almost everywhere to ensure the change of variables formula is valid.

Normalizing Flow Models:

Jacobian (2D Case)

- ▶ The Jacobian is a matrix of partial derivatives:

$$J_{ij} = \frac{\partial f_i}{\partial z_j}$$

- ▶ It describes how each component of $f(z)$ changes with z .
- ▶ The Jacobian is needed to compute how volumes change after transformation.

$$1) \quad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$$

$$x = f(z) \quad z = f^{-1}(x)$$

$$2) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} z_1 + z_2 \\ 2z_1 \end{bmatrix} = f \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

$$\begin{bmatrix} x_2/2 \\ x_1 - x_2/2 \end{bmatrix} = f^{-1} \left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right)$$

$$3) \quad J_f = \begin{bmatrix} \overbrace{\partial x_1 / \partial z_1}^{\text{input}} & \partial x_1 / \partial z_2 \\ \partial x_2 / \partial z_1 & \underbrace{\partial x_2 / \partial z_2}_{\text{output}} \end{bmatrix}$$

$$J_{f^{-1}} = \begin{bmatrix} \partial z_1 / \partial x_1 & \partial z_1 / \partial x_2 \\ \partial z_2 / \partial x_1 & \partial z_2 / \partial x_2 \end{bmatrix}$$

$$4) \quad J_f = \begin{bmatrix} 1 & 1 \\ 2 & 0 \end{bmatrix}$$

$$J_{f^{-1}} = \begin{bmatrix} 0 & 1/2 \\ 1 & -1/2 \end{bmatrix}$$

$$J_f J_{f^{-1}} = I$$

15

Example of a 2D Jacobian matrix. See the Appendix for more information.

Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 1, slide 19; after Hung-yi Lee, "Flow-based Generative Model" (NTU)

The determinant of a **square matrix** is a **scalar** that provides information about the matrix.

• 2 x 2

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$\det(A) = ad - bc$$

$$\det(A) = 1/\det(A^{-1})$$

$$\det(J_f) = 1/\det(J_{f^{-1}})$$

• 3 x 3

$$A = \begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix}$$

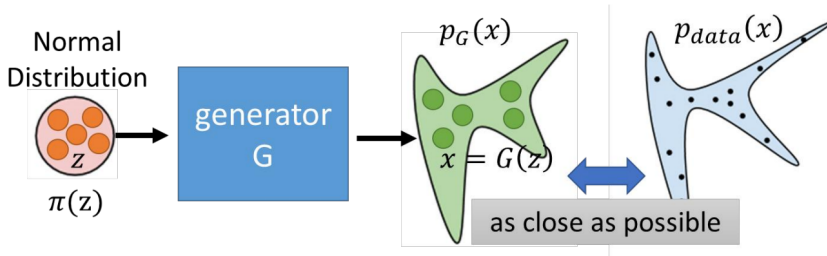
$$\det(A) =$$

$$a_1a_5a_9 + a_2a_6a_7 + a_3a_4a_8 \\ - a_3a_5a_7 - a_2a_4a_9 - a_1a_6a_8$$

Example of matrix determinant.

Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 1, slide 20; after Hung-yi Lee, "Flow-based Generative Model" (NTU)

- ▶ A generator G is a network that maps a simple distribution (for example, a normal distribution) $\pi(z)$ to a complex data distribution $p_G(x)$, which aims to be as close as possible to the real data distribution $p_{\text{data}}(x)$.

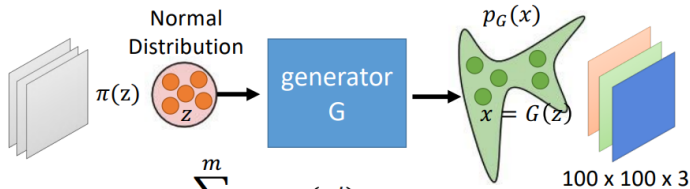


The goal of a generator network in a generative model

Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 1, slide 7; after Hung-yi Lee, "Flow-based Generative Model" (NTU)

- ▶ Consider a directed latent-variable model with observed variables X and latent variables Z . In a normalizing flow model, the mapping between Z and X is deterministic and invertible: $G : \mathbb{R}^n \rightarrow \mathbb{R}^n$, such that $X = G(Z)$ and $Z = G^{-1}(X)$.
- ▶ By the change of variables formula, the marginal likelihood $p(x)$ is:

$$p_X(x; \theta) = p_Z(G_\theta^{-1}(x)) \left| \det \left(\frac{\partial G_\theta^{-1}(x)}{\partial x} \right) \right|$$



$$G^* = \arg \max_G \sum_{i=1}^m \log p_G(x^i)$$

$$p_G(x^i) = \pi(z^i) |\det(J_{G^{-1}})|$$

$$z^i = G^{-1}(x^i)$$

You can compute $\det(J_G)$

You know G^{-1}

G has limitation

$$\log p_G(x^i) = \log \pi(G^{-1}(x^i)) + \log |\det(J_{G^{-1}})|$$

Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 1, slide 27; after Hung-yi Lee, "Flow-based Generative Model" (NTU)

- ▶ The expressiveness of G is limited. To model more complex distributions, we need more expressive generators.
- ▶ Composing K invertible layers $G_1, \dots, G_K$, so that $z = G_1^{-1}(\dots G_K^{-1}(x))$, gives

$$\log p_K(x) = \log \pi(z) + \sum_{h=1}^K \log \left| \det \left(J_{G_h^{-1}} \right) \right|$$



$$p_1(x^i) = \pi(z^i) \left(\left| \det(J_{G_1^{-1}}) \right| \right) \quad z^i = G_1^{-1}(\dots G_K^{-1}(x^i))$$

$$p_2(x^i) = \pi(z^i) \left(\left| \det(J_{G_1^{-1}}) \right| \right) \left(\left| \det(J_{G_2^{-1}}) \right| \right)$$

$\vdots$

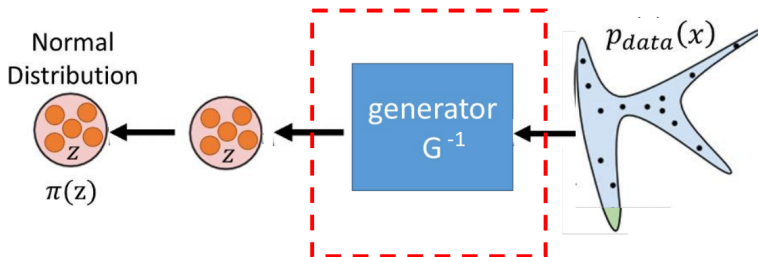
$$p_K(x^i) = \pi(z^i) \left(\left| \det(J_{G_1^{-1}}) \right| \right) \dots \left(\left| \det(J_{G_K^{-1}}) \right| \right)$$

$$\log p_K(x^i) = \log \pi(z^i) + \sum_{h=1}^K \log \left| \det(J_{G_h^{-1}}) \right| \quad \text{Maximise}$$

Corrections: the base density is $\pi(z)$, not $\pi(x)$, and the summand in the last line should read $\log \left| \det \left(J_{G_h^{-1}} \right) \right|$, matching the index of the sum.

Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 1, slide 28; after Hung-yi Lee, "Flow-based Generative Model" (NTU)

- In practice, we train G^{-1} , but use G for generation.



Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 1, slide 29; after Hung-yi Lee, "Flow-based Generative Model" (NTU)

- ▶ **Normalizing**: The change of variables produces a normalized density after applying an invertible transformation.
- ▶ **Flow**: Invertible transformations can be composed to create more complex, expressive mappings.

Normalizing Flow Models: **Learning and Inference**

- ▶ Learning is performed via maximum likelihood estimation over the dataset D :

$$\max_{\theta} \log p(D; \theta) = \sum_{x \in D} \left[\log \pi(G_{\theta}^{-1}(x)) + \log \left| \det \left(\frac{\partial G_{\theta}^{-1}(x)}{\partial x} \right) \right| \right]$$

- ▶ Exact likelihood evaluation is achieved using the inverse transformation and the change of variables formula.
- ▶ Sampling is performed via the forward transformation $G_{\theta} : Z \rightarrow X$:

$$z \sim \pi(z), \quad x = G_{\theta}(z)$$

- ▶ Latent representations are inferred via the inverse transformation (no inference network required):

$$z = G_{\theta}^{-1}(x)$$

Normalizing Flow Models:

Requirements

- ▶ A simple prior $\pi(z)$ that allows for efficient sampling and tractable likelihood evaluation (e.g., Gaussian).
- ▶ Invertible transformations.
- ▶ Computing likelihoods also requires evaluating the determinants of $n \times n$ Jacobian matrices, where n is the data dimensionality.
 - Computing the determinant of an $n \times n$ matrix is $O(n^3)$, which is prohibitively expensive within a learning loop.
 - Key idea: Choose transformations so that the resulting Jacobian matrix has a special structure. For example, the determinant of a triangular matrix is the product of its diagonal entries, making it an $O(n)$ operation.

- ▶ We need a fast and simple prior (e.g., Gaussian).
- ▶ The big question: How do we obtain invertible transformations and efficiently compute the determinants of Jacobians?
- ▶ **One solution:** Use coupling layers to design invertible functions and efficiently calculate the determinant of the Jacobian!

Normalizing Flow Models:

Coupling layers

Concept:

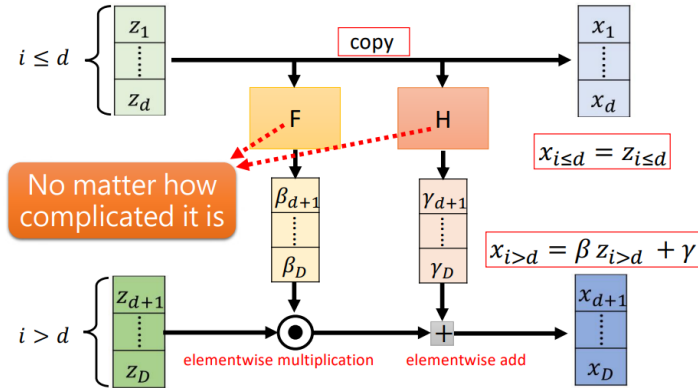
- ▶ Split the input z into two parts: $z = [z_{1:d}, z_{d+1:n}]$.
- ▶ Copy one part through unchanged and transform the other conditioned on it:

$$\begin{aligned}x_{1:d} &= z_{1:d} \\x_{d+1:n} &= \beta \odot z_{d+1:n} + \gamma, \quad \beta = F(z_{1:d}), \quad \gamma = H(z_{1:d})\end{aligned}$$

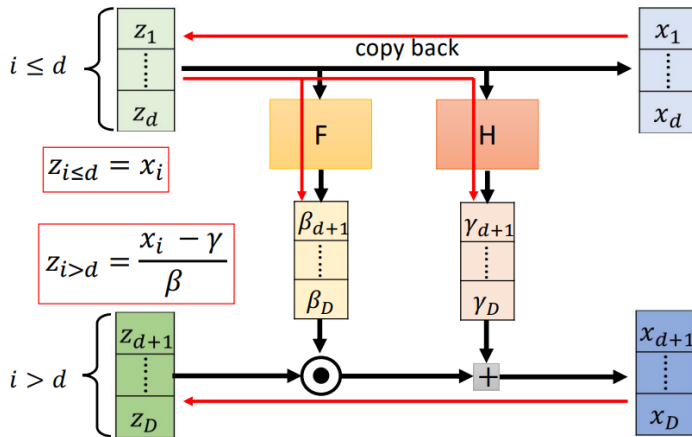
where F and H are the scale and translation networks. RealNVP will instead have F output the *log-scale*, $\beta = \exp(F(z_{1:d}))$, which keeps β positive.

Advantages:

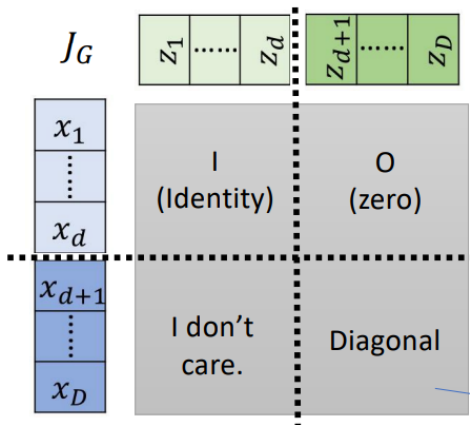
- ▶ Invertibility is straightforward.
- ▶ The Jacobian is triangular, making determinant computation efficient.
- ▶ Facilitates the design of complex, yet tractable, transformations.



Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 2, slide 7; after Hung-yi Lee, "Flow-based Generative Model" (NTU)



Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 2, slide 8; after Hung-yi Lee, "Flow-based Generative Model" (NTU)



$$\det(J_G)$$

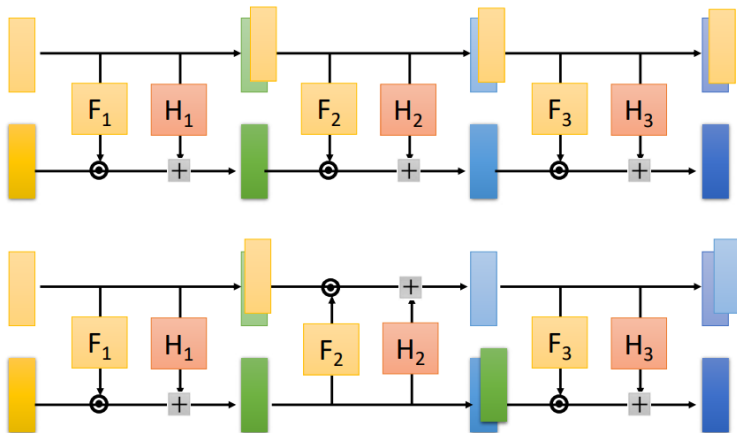
$$= \frac{\partial x_{d+1}}{\partial z_{d+1}} \frac{\partial x_{d+2}}{\partial z_{d+2}} \dots \frac{\partial x_D}{\partial z_D}$$

$$= \beta_{d+1} \beta_{d+2} \dots \beta_D$$

$$x_{i>d} = \beta z_{i>d} + \gamma$$

Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 2, slide 10; after Hung-yi Lee, "Flow-based Generative Model" (NTU)

Coupling layers (cont.)

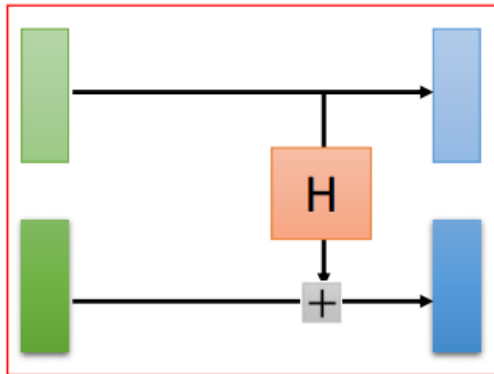


Source: Hao Dong, "Deep Generative Models" (Peking University), Normalizing Flow Models Part 2, slide 12; after Hung-yi Lee, "Flow-based Generative Model" (NTU)

Normalizing Flow Models:

NICE: Non-linear Independent Components Estimation

- ▶ **Overview:** Introduced by Dinh et al. (2014), NICE employs additive coupling layers for constructing invertible transformations.
- ▶ **Additive Coupling Layers:**
 - Partition the variables z into two disjoint subsets:
 - $x_{1:d} = z_{1:d}$
 - $x_{d+1:n} = z_{d+1:n} + H(z_{1:d})$, where H is a neural network.
 - The Jacobian determinant of this transformation is 1, which simplifies density computation.

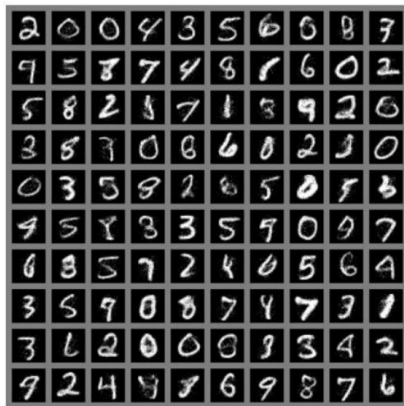


Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 2, slide 15

- ▶ Multiple additive coupling layers are composed together, with different partitions in each layer.
- ▶ **Limitation:** The transformation is volume-preserving and cannot model changes in volume, which limits expressiveness.
- ▶ **Enhancement:** A final rescaling layer is applied to introduce volume changes:

$$x = s \odot z$$

where s is a scaling factor.



NICE generated samples when trained on the MNIST digits dataset.

Source: Dinh et al., "NICE: Non-linear Independent Components Estimation," arXiv:1410.8516, 2014, Fig. 5(a).



NICE generated samples when trained on the CIFAR-10 dataset.

Source: Dinh et al., "NICE: Non-linear Independent Components Estimation," arXiv:1410.8516, 2014, Fig. 5(d).

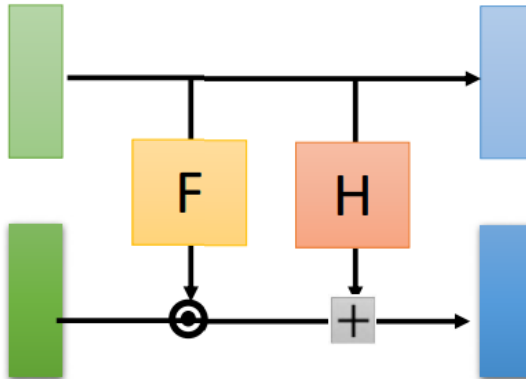
Normalizing Flow Models:

Real NVP - Real-valued Non-Volume Preserving

RealNVP: Real-valued Non-Volume Preserving

► Enhancements over NICE:

- Introduces scaling in coupling layers:
 - Partition the variables z into two disjoint subsets.
 - $x_{1:d} = z_{1:d}$
 - $x_{d+1:n} = z_{d+1:n} \odot \exp(F(z_{1:d})) + H(z_{1:d})$, where F and H are neural networks. Writing the scale as $\beta = \exp(F(z_{1:d}))$ keeps it positive.
- Allows modeling of volume changes, increasing flexibility.



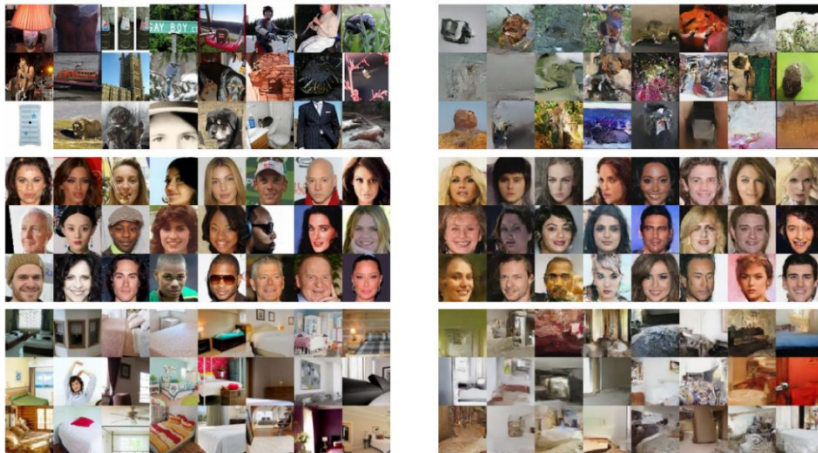
Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 2, slide 20

► Benefits:

- Efficient computation of the Jacobian determinant due to the triangular structure:
 $\log |\det J_G| = \sum_{i=d+1}^n F_i(z_{1:d}).$
- Supports exact likelihood estimation and sampling.

► Implementation Details:

- Alternating the roles of $z_{1:d}$ and $z_{d+1:n}$ across layers ensures all dimensions are transformed.
- Coupling layers are composed together (with arbitrary partitions of variables in each layer).



Real NVP: dataset examples (left of each pair) vs. model samples (right). Rows: ImageNet 64x64, CelebA, LSUN bedroom (Dinh et al., 2016).

Normalizing Flow Models:

Glow: Generative Flow

Innovations:

Introduced by Kingma and Dhariwal (2018), Glow incorporates:

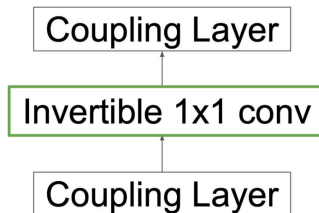
- ▶ **Invertible 1×1 convolutions** for channel mixing.
- ▶ **ActNorm layers** for per-channel normalization with data-dependent initialization.
- ▶ **Affine coupling layers** similar to RealNVP.

Advantages:

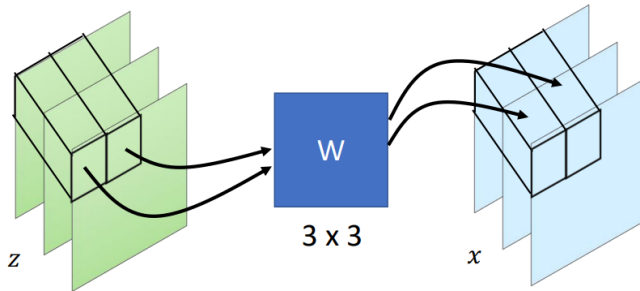
- ▶ Improved expressiveness and stability.
- ▶ Enhanced performance in image generation tasks.

Key Components:

- ▶ **ActNorm:** Applies per-channel affine transformation initialized with data statistics.
- ▶ **Invertible 1×1 Convolution:** Generalizes permutation operations, allowing learned channel mixing.
- ▶ **Affine Coupling Layers:** As in RealNVP, but integrated with the above components for greater flexibility.



Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 2, slide 23



W can shuffle the channels.

If W is invertible, it is easy to compute W^{-1} .

3
1
2

=

0	0	1	1
1	0	0	2
0	1	0	3

Source: Hao Dong, "Deep Generative Models" (Peking University), Normalising Flow Models Part 2, slide 24; after Hung-yi Lee, "Flow-based Generative Model" (NTU)



Glow generated samples

Source: Kingma & Dhariwal, "Glow: Generative Flow with Invertible 1x1 Convolutions," NeurIPS 2018, Fig. 1.



Linear interpolation in latent space between real images with Glow

Source: Kingma & Dhariwal, "Glow: Generative Flow with Invertible 1x1 Convolutions," NeurIPS 2018, Fig. 5.

Normalizing Flow Models:

Flow Matching

- ▶ So far: a flow is a **stack of invertible layers**, each designed so its Jacobian determinant is cheap (coupling layers, 1×1 convolutions).
- ▶ **Continuous normalizing flows (CNFs)** (Chen et al., 2018) replace the stack with a **vector field**: a network $v_\theta(x, t)$ defines an ODE

$$\frac{dx_t}{dt} = v_\theta(x_t, t),$$

and the flow is the result of integrating it from $t = 0$ to $t = 1$.

- ▶ ODE solutions are invertible by construction (integrate backwards), so **any architecture** can be used: no coupling-layer constraints.

- ▶ The change of variables formula becomes **instantaneous**:

$$\frac{\partial \log p(x_t)}{\partial t} = -\text{tr} \left(\frac{\partial v_\theta}{\partial x_t} \right),$$

a trace instead of a log-determinant.

- ▶ **The catch:** training a CNF by maximum likelihood (FFJORD, Grathwohl et al., 2019) requires **simulating the ODE** and estimating the trace at every training step.
- ▶ Accurate and architecture-free, but slow and hard to scale to large images. This is the problem flow matching removes.

- ▶ **Flow Matching (FM)** (Lipman et al., ICLR 2023) trains the same object, a vector field v_θ , **without ever simulating the ODE**.
- ▶ Idea: fix a **target probability path** p_t that moves from noise ($t = 0$) to data ($t = 1$), with generating field u_t , and simply **regress**:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, x \sim p_t} \|v_\theta(x, t) - u_t(x)\|^2.$$

- ▶ Problem: u_t for the whole data distribution is unknown in closed form, so this loss cannot be computed directly.

- ▶ Solution: define the path **per sample pair**: condition on a noise sample x_0 and a data sample x_1 , giving a conditional path $p_t(x | x_0, x_1)$ whose field $u_t(x | x_0, x_1)$ is known, and regress on that instead:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, x_0, x_1, x \sim p_t(x|x_0, x_1)} \|v_\theta(x, t) - u_t(x | x_0, x_1)\|^2.$$

- ▶ **Key theorem:** $\mathcal{L}_{\text{CFM}}$ and $\mathcal{L}_{\text{FM}}$ have the **same gradients**, so the tractable conditional objective trains the true marginal field.
- ▶ Training is now plain **supervised regression**: no ODE solver, no Jacobians, no invertibility constraints on the architecture.

- Choose the conditional path that **linearly interpolates** between a noise sample and a data sample:

$$x_t = (1 - t) x_0 + t x_1, \quad x_0 \sim \mathcal{N}(0, I), \quad x_1 \sim p_{\text{data}},$$

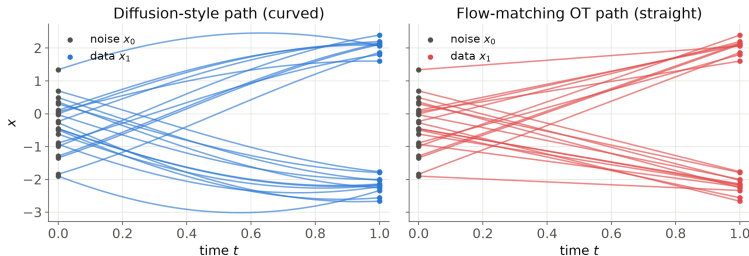
whose velocity is constant along the path: $u_t = x_1 - x_0$.

- The full training step:

1. Sample $t \sim \mathcal{U}[0, 1]$, noise x_0 , and a data point x_1 .
2. Form x_t and take one gradient step on $\|v_\theta(x_t, t) - (x_1 - x_0)\|^2$.

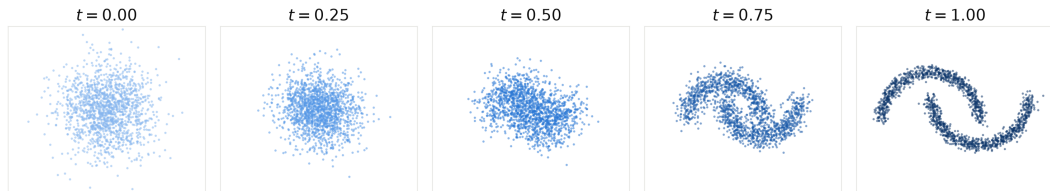
- This linear choice is the **rectified flow** of Liu et al. (2023) and the linear **stochastic interpolant** of Albergo & Vanden-Eijnden (2023).
- **Sampling:** integrate $dx/dt = v_\theta(x, t)$ from noise at $t = 0$ to $t = 1$; straighter paths need fewer steps.

Same endpoints, different probability paths



- Diffusion-style (variance-preserving) paths take a **curved detour** between the same endpoints; the optimal-transport / linear paths of flow matching go **straight**.
- Straight paths are easy to integrate: a few Euler steps suffice, which is why flow-matching models sample fast.

Samples along the straight path $x_t = (1 - t)x_0 + tx_1$: noise $\rightarrow$ two moons



- ▶ Snapshots of the straight path carrying Gaussian noise into a two-moons distribution: the network v_θ learns the **velocity field** that produces exactly this motion.
- ▶ At inference, start from fresh noise and follow the learned field; likelihoods remain available through the instantaneous change of variables from the earlier slide.

- ▶ **Classical flows (this lecture):** discrete invertible layers, exact likelihood, but constrained architectures and many layers for expressiveness.
- ▶ **CNF with maximum likelihood:** free architecture, but training requires ODE simulation.
- ▶ **Diffusion models (covered later):** learn to reverse a gradual noising process; sampling follows a **curved** path and traditionally needs many steps.
- ▶ **Flow matching:** keeps the free architecture, trains by regression (no simulation), and samples with a deterministic ODE along **straighter** paths.
- ▶ The families are close relatives: the diffusion (variance-preserving) path is one particular choice of probability path in the flow-matching framework, and score-based training corresponds to a different parameterization of the same object.

- ▶ **Stable Diffusion 3** (Esser et al., 2024): a rectified-flow transformer. Its large-scale ablation found that a rectified-flow path beats established diffusion formulations only once the timestep sampling is biased towards intermediate t (logit-normal sampling); with uniform t it does not.
- ▶ **FLUX.1** (Black Forest Labs, 2024): open-weight rectified-flow transformer, among the strongest open text-to-image models at release.
- ▶ **Meta Movie Gen** (2024): flow matching as the training objective for large-scale video generation.
- ▶ **Voicebox** (Meta, 2023): flow matching for speech generation and editing.
- ▶ Tooling: Meta's `flow_matching` library and the “Flow Matching Guide and Code” (Lipman et al., 2024) give a reference implementation of the whole framework.

- ▶ Flow matching is the **continuous-time revival** of everything in this lecture: the same change-of-variables machinery, with the discrete stack replaced by a learned velocity field.
- ▶ Training reduces to **regression on interpolations** between noise and data: simple to implement, stable, and free of architectural constraints.
- ▶ Straight probability paths give **fast sampling**; likelihoods stay tractable through the instantaneous change of variables.
- ▶ Normalizing flows did not disappear: reborn as flow matching, they are the objective behind several of today's strongest image, video, and speech generators.

Key Takeaways:

- ▶ Normalizing flows model complex distributions while keeping likelihoods exact.
- ▶ They transform simple base distributions into complex data distributions using invertible mappings.
- ▶ The change of variables formula and the Jacobian determinant are central to their mathematical foundation.
- ▶ Architectures such as NICE, RealNVP, and Glow illustrate the evolution and increasing sophistication of flow-based models.
- ▶ Flow matching revives flows in continuous time: train a velocity field by regression on noise-data interpolations, then sample by integrating an ODE; it powers current state-of-the-art image, video, and speech generators.

Considerations:

- ▶ There are trade-offs between model expressiveness and computational efficiency.
- ▶ Efficient learning and density estimation both depend on designing transformations with tractable Jacobians.

Normalizing Flow Models:

Limitations

- ▶ **Invertibility constrains the architecture:** every layer must be invertible, which rules out most standard building blocks.
- ▶ **The determinant must stay cheap:** coupling layers make it $O(n)$, but only by restricting each layer to transform half its input at a time, so many layers are needed for expressiveness. These first two constraints apply to discrete-layer flows; flow matching drops both.
- ▶ **No dimensionality reduction:** the change of variables formula needs $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$, so the latent space is exactly as large as the data. Unlike a VAE, a flow has no bottleneck and cannot compress.
- ▶ **Continuous data only:** the formula assumes continuous densities, so discrete values such as 8-bit pixels must first be dequantized by adding noise. RealNVP then applies a logit transform, $\text{logit}(\alpha + (1 - \alpha)x/256)$ with $\alpha = 0.05$, so that the density is not pushed against the boundaries of the pixel range.
- ▶ In practice flows can trail autoregressive models on likelihood, and large ones are slow.

Normalizing Flow Models:

References

Reference Slides

- ▶ Fei-Fei Li, “Deep Learning for Computer Vision” (Stanford CS231n)
- ▶ Stefano Ermon, “Deep Generative Models” (Stanford CS236)
- ▶ Hao Dong, “Deep Generative Models” (Peking University)
- ▶ Hung-yi Lee, “Machine Learning” (National Taiwan University)
- ▶ Murtaza Taj, “Deep Learning” (CS437)

- [1] Dinh, L., Krueger, D., & Bengio, Y. (2014). NICE: Non-linear Independent Components Estimation. *arXiv preprint arXiv:1410.8516*.
- [2] Dinh, L., Sohl-Dickstein, J., & Bengio, S. (2016). Density estimation using Real NVP. *arXiv preprint arXiv:1605.08803*.
- [3] Kingma, D. P., & Dhariwal, P. (2018). Glow: Generative flow with invertible 1x1 convolutions. *Advances in Neural Information Processing Systems*, 31.
- [4] Rezende, D. J., & Mohamed, S. (2015). Variational inference with normalizing flows. *International Conference on Machine Learning (ICML)*, 1530-1538.
- [5] Weng, L. (2018). Flow-based Deep Generative Models.
<https://lilianweng.github.io/posts/2018-10-13-flow-models/>

- [6] Kobyzev, I., Prince, S. J. D., & Brubaker, M. A. (2021). Normalizing Flows: An Introduction and Review of Current Methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(11), 3964-3979.
- [7] Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. (2018). Neural Ordinary Differential Equations. *Advances in Neural Information Processing Systems*, 31.
- [8] Grathwohl, W., Chen, R. T. Q., Bettencourt, J., Sutskever, I., & Duvenaud, D. (2019). FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models. *International Conference on Learning Representations (ICLR)*.
- [9] Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M. (2023). Flow Matching for Generative Modeling. *International Conference on Learning Representations (ICLR)*. arXiv:2210.02747.

- [10] Liu, X., Gong, C., & Liu, Q. (2023). Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. *International Conference on Learning Representations (ICLR)*. arXiv:2209.03003.
- [11] Albergo, M. S., & Vanden-Eijnden, E. (2023). Building Normalizing Flows with Stochastic Interpolants. *International Conference on Learning Representations (ICLR)*. arXiv:2209.15571.
- [12] Esser, P., et al. (2024). Scaling Rectified Flow Transformers for High-Resolution Image Synthesis (Stable Diffusion 3). *International Conference on Machine Learning (ICML)*. arXiv:2403.03206.
- [13] Black Forest Labs (2024). FLUX.1: [pro], [dev] and [schnell].
<https://bfl.ai/announcing-black-forest-labs>
- [14] Polyak, A., et al. (2024). Movie Gen: A Cast of Media Foundation Models. *arXiv preprint arXiv:2410.13720*.

- [15] Le, M., Vyas, A., Shi, B., et al. (2023). Voicebox: Text-Guided Multilingual Universal Speech Generation at Scale. *Advances in Neural Information Processing Systems*, 36.
- [16] Lipman, Y., et al. (2024). Flow Matching Guide and Code. *arXiv preprint arXiv:2412.06264*. Code: https://github.com/facebookresearch/flow_matching.

Normalizing Flow Models:

Appendix

Jacobian Matrix (2D Case)

Intuition Behind the Jacobian: In multivariable calculus, the Jacobian matrix describes how a transformation function changes space locally. Think of it as a local linear approximation to a nonlinear transformation. In 2D, it tells us how small changes in the input (x, y) affect the output (u, v) .

Formal Definition: Let

$$f(x, y) = (u(x, y), v(x, y))$$

be a transformation from $\mathbb{R}^2 \rightarrow \mathbb{R}^2$. The Jacobian matrix is:

$$J_f(x, y) = \begin{bmatrix} \frac{\partial u}{\partial x} & \frac{\partial u}{\partial y} \\ \frac{\partial v}{\partial x} & \frac{\partial v}{\partial y} \end{bmatrix}$$

Each entry measures how one output dimension changes with respect to one input dimension.

Geometric Interpretation:

The determinant of the Jacobian, $|\det J_f(x, y)|$, tells us how much area is stretched or compressed by the transformation at a specific point.

- ▶ $|\det J| > 1$: area expands.
- ▶ $|\det J| < 1$: area contracts.
- ▶ $|\det J| = 0$: local collapse (not invertible).

Example (Shear + Scaling): Let

$$f(x, y) = (2x + y, x + 3y)$$

Then the Jacobian is:

$$J_f(x, y) = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$$

Determinant:

$$\det J = 2 \cdot 3 - 1 \cdot 1 = 6 - 1 = 5$$

Interpretation: Locally, this transformation scales area by a factor of 5.

In normalizing flows, we use:

$$p_X(x) = p_Z(f^{-1}(x)) \cdot \left| \det \left(\frac{\partial f^{-1}}{\partial x} \right) \right|$$

To compute the exact density after transformation, we must evaluate the Jacobian determinant. Hence, choosing transformations where the Jacobian (or its determinant) is easy to compute is critical (e.g., triangular matrices in coupling layers).

Visual Aid: For an interactive visualization of how the Jacobian affects area in 2D transformations, visit the Wolfram Demonstrations Project: